

Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



DISCRETE STRUCTURE (CO1007)

Assignment Report

A* ALGORITHMS

Instructor(s): Nguyễn An Khương, *CSE-HCMUT*
Lê Hồng Trang, *CSE-HCMUT*
Trần Tuấn Anh, *CSE-HCMUT*
Phạm An Vinh, *CSE-HCMUT*
Mai Xuân Toàn, *CSE-HCMUT*
Trần Hồng Tài, *CSE-HCMUT*
Class: CC02
Student: Hắc Minh Quân, *2550201*

Contents

1	Citation Convention	3
2	Introduction	3
3	Theoretical Background	3
3.1	Graphs, paths, and shortest paths	3
3.2	Uninformed-to-Informed search	4
3.3	A* evaluation function	4
3.4	Admissibility and consistency	4
3.5	Generic A*	5
4	Implementation Architecture	5
4.1	Output structure	5
4.2	Validation program	6
5	Task 1: Degrees of Separation in a Social Network	6
5.1	Model	6
5.2	Reverse-BFS heuristic	6
5.3	Manual trace	7
5.4	Analysis	7
6	Task 2: Drone Delivery in 2D Space with Three Heuristics	7
6.1	Model	7
6.2	Three heuristic	8
6.3	Open-set scan, relaxation, and reconstruction	8
6.4	Test case: Aurora relay	9
6.5	Unreachable case	10
7	Task 3: Warehouse Robot Navigation with Obstacles	10
7.1	Model	10
7.2	Implementation-level analysis	10
7.3	Test case: Diagonal keyhole	11
7.4	Test case: Sealed freezer vault	12
8	Task 4: Evacuation Route Planning	12
8.1	Model and grid-to-matrix transformation	12
8.2	A* over vertex indices	12



8.3	Test cases	13
9	Comparative Heuristic Discussion	13
10	Correctness Argument	14
10.1	Validity of returned paths	14
10.2	Optimality under standard assumptions	14
10.3	Implementation limitations affecting proofs	14
11	Complexity Analysis	14
12	Testing and Demonstration	15
13	Limitations and Future Improvements	16
13.1	Limitations	16
13.2	Future work	16
14	Conclusion	16
	References	17

1 Citation Convention

Table 1: Code artifact IDs used in the report.

ID	File	Purpose
C1	PathNode.h	Declares the shared linked-list output node.
C2	PathNode.cpp	Defines the required path printer.
C3	Algo.h	Declares the four auto-tested function signatures.
C4	Algo.cpp	Implements all four path-finding tasks and helper functions.
C5	main.cpp	Builds demonstration and validation scenarios.
T1	test_cases/expected_output_from_main.txt	Exact console output generated from the validation program.

2 Introduction

This report studies A* search through a concrete C++ implementation for CO1007 Discrete Structure. The project applies the evaluation function $f(n) = g(n) + h(n)$ to four path-finding models: social degrees of separation, drone delivery over a weighted coordinate graph, warehouse robot navigation with obstacles, and evacuation planning after converting a floor plan into a weight matrix. The discussion combines theory and implementation evidence: it cites classical graph-search literature for background, including Hart, Nilsson, and Raphael’s original A* paper [1], Dijkstra’s shortest-path work [2], and standard algorithms and AI references [3, 4, 5, 6]; it also cites the submitted source code by artifact and line range, such as [C4, ll. 98-107] for the drone heuristics and [C4, ll. 340-372] for the evacuation grid-to-matrix conversion. The result is a self-contained, IEEE-style report with figures, manual traces, correctness arguments, complexity analysis, and validation evidence from the `main.cpp` scenarios.

3 Theoretical Background

3.1 Graphs, paths, and shortest paths

A graph is a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges. In a weighted graph, each edge $(u, v) \in E$ is assigned a non-negative cost $w(u, v)$. This mathematical model is central to discrete mathematics and algorithm design because it abstracts transportation networks, communication systems, social links, indoor maps, and state spaces [5, 3]. A path from source s to target t is a sequence

$$P = (v_0, v_1, \dots, v_k), \quad v_0 = s, \quad v_k = t,$$

where every consecutive pair is an edge. Its weighted cost is

$$\text{cost}(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$

The shortest-path problem asks for a path of minimum cost. Dijkstra's algorithm solves the non-negative single-source shortest-path problem without a goal-directed heuristic [2, 3]. A* can be viewed as a goal-directed extension of uniform-cost search and Dijkstra-style reasoning, because it combines the path cost already paid with an estimate of the cost still required [1, 4].

3.2 Uninformed-to-Informed search

In uninformed search, the algorithm does not know where the goal is except through explicit goal tests. Breadth-first search is optimal for unweighted graphs, while Dijkstra's algorithm is optimal for non-negative weighted graphs. However, both may explore large areas unrelated to the goal. Informed search introduces a heuristic $h(n)$, a problem-specific estimate of the remaining distance. The purpose of a good heuristic is not merely to be mathematically elegant; it changes the geometry of the search. For a drone in two-dimensional space, a Euclidean heuristic acts like a compass. For a warehouse robot allowed to move diagonally, a Chebyshev-like heuristic expresses diagonal reachability. For a social graph, a reverse breadth-first traversal gives exact remaining hop count.

3.3 A* evaluation function

The defining equation of A* is

$$f(n) = g(n) + h(n), \quad (1)$$

where $g(n)$ is the best known cost from the start to n , $h(n)$ estimates the cost from n to the goal, and $f(n)$ estimates the total solution cost through n . Hart, Nilsson, and Raphael introduced the formal basis for this evaluation in the context of heuristic path determination [1]. Russell and Norvig present A* as a canonical informed-search algorithm and emphasize the role of admissibility and consistency in guaranteeing optimality [4].

3.4 Admissibility and consistency

A heuristic is admissible if it never overestimates the true optimal remaining cost:

$$0 \leq h(n) \leq d^*(n, t), \quad (2)$$

where $d^*(n, t)$ is the optimal cost from n to target t . It is consistent if for every edge (u, v) ,

$$h(u) \leq w(u, v) + h(v). \quad (3)$$

Consistency is stronger and is especially useful because once a node is expanded under a consistent heuristic, its best cost is final [1, 6]. This report uses those notions carefully. For example, the Task 1 social heuristic is exact and therefore admissible. In contrast, the Task 3 Manhattan heuristic can overestimate under diagonal movement with cost 1.5: from $(0, 0)$ to $(1, 1)$, Manhattan gives 2 but a diagonal move costs 1.5.

3.5 Generic A*

Algorithm 1 Generic A* search

```

1: Set  $g(s) = 0$ ,  $f(s) = h(s)$ ; set all other  $g$ -values to  $\infty$ .
2: Insert  $s$  into the open set.
3: while the open set is not empty do
4:   Select a node  $u$  with minimum  $f(u)$ .
5:   if  $u$  is the goal then
6:     reconstruct the path by following parent pointers and return it.
7:   end if
8:   Move  $u$  to the closed set.
9:   for each neighbor  $v$  of  $u$  do
10:    Let  $\hat{g} = g(u) + w(u, v)$ .
11:    if  $\hat{g} < g(v)$  then
12:      update  $g(v) = \hat{g}$ ,  $f(v) = g(v) + h(v)$ , and  $\text{parent}(v) = u$ .
13:    end if
14:  end for
15: end while
16: Return failure.

```

The submitted code follows this scheme most directly in Tasks 2-4. It uses arrays and vectors rather than priority queues, which keeps the implementation compatible with the assignment environment but gives quadratic worst-case behavior.

4 Implementation Architecture

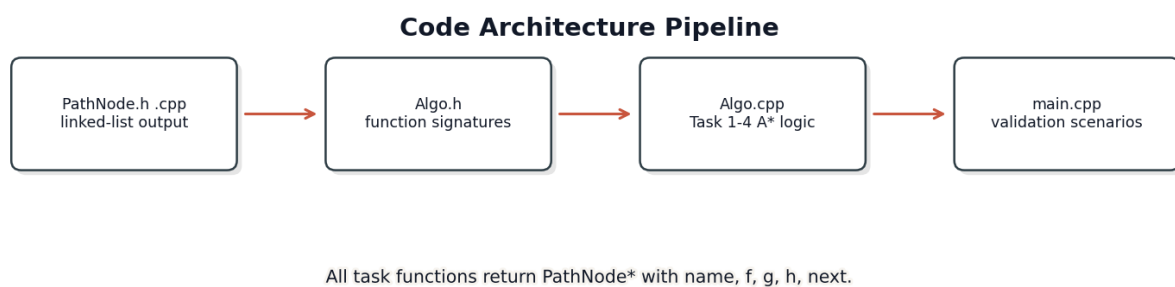


Figure 1: Four task functions return a shared PathNode* linked list.

4.1 Output structure

The file PathNode.h declares the common output node containing name, f, g, h, and next [C1, ll. 6-15]. The corresponding printer in PathNode.cpp emits each node in the required format [C2, ll. 6-17]. This design gives all tasks a uniform output even though their labels differ: person IDs in Task 1, coordinate strings in Task 2, move directions in Task 3, and grid coordinates in Task 4.

4.2 Validation program

The modified `main.cpp` is no longer a trivial driver. It contains helper routines for freeing paths, printing cases, connecting undirected weighted edges, loading ASCII grids, and displaying maps [C5, ll. 4-80]. It then constructs named validation scenarios: the friend bridge, Aurora relay, blackout island, diagonal keyhole, sealed freezer vault, fire-door staircase, smoke-ring floor, and zero-movement evacuation [C5, ll. 100-308]. These names are useful because they turn test cases into interpretable experiments rather than anonymous data.

5 Task 1: Degrees of Separation in a Social Network

5.1 Model

The first task models a social network as an adjacency matrix. A vertex is a person, and an edge represents a direct social connection. The cost of every connection is one, so the optimal path is the path with the fewest social hops. The implementation uses

$$g(n) = \text{number of connections from the start to } n.$$

5.2 Reverse-BFS heuristic

The most interesting feature of Task 1 is that the heuristic is not guessed. The function `computeMinStepsToGoal` initializes all heuristic values to infinity, sets the goal heuristic to zero, and performs a reverse traversal over incoming adjacency-matrix entries [C4, ll. 52-78]. Thus, $h(n)$ is the exact minimum number of social connections from node n to the goal. Exact distance is both admissible and consistent because it satisfies the triangle inequality on the unweighted graph.

After computing this heuristic, `findSocialPath` passes it to `aStarShortestPath` [C4, ll. 80-87]. The latter does not use a general open set. It greedily follows a neighbor whose heuristic is exactly one smaller than the current node's heuristic [C4, ll. 20-33]. Because the heuristic is exact, this descent preserves optimality: each step moves one layer closer to the goal in the shortest-path distance map.

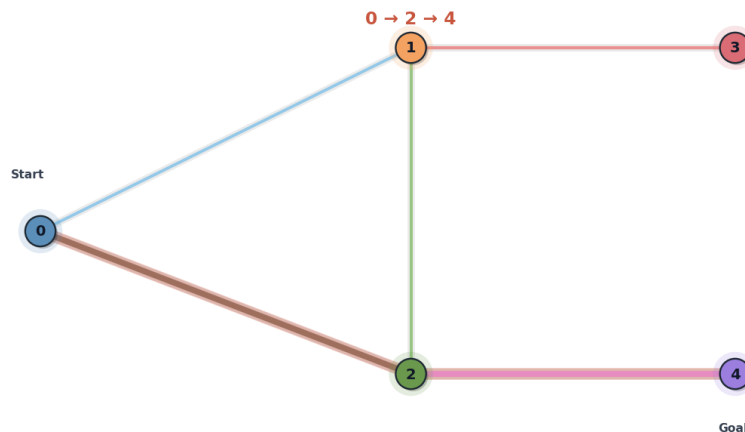


Figure 2: The exact shortest connection path from person 0 to person 4

5.3 Manual trace

The modified driver constructs the graph in Fig. 2 and calls `findSocialPath(social, 0, 4)` [C5, ll. 100-130]. The generated output is recorded in [T1]. Table 2 gives the manual trace.

Table 2: Task 1 manual A* trace for the friend bridge test.

Step	Node	$g(n)$	$h(n)$	$f(n)$	Decision
0	0	0	2	2	Start. Reverse BFS says node 0 is two hops away from node 4.
1	2	1	1	2	Select node 2 because its heuristic is exactly one smaller.
2	4	2	0	2	Goal reached. Output path: $0 \rightarrow 2 \rightarrow 4$.

5.4 Analysis

This task is a compact example of a perfect heuristic: $h(n) = d^*(n, t)$. With such a heuristic, the search does not waste work expanding off-path nodes. The limitation is that computing an exact heuristic already solves the unweighted distance problem from the goal. This is acceptable for the assignment and pedagogically valuable, but in larger or weighted social graphs a full A* or Dijkstra implementation would be more general.

6 Task 2: Drone Delivery in 2D Space with Three Heuristics

6.1 Model

The second task uses a weighted graph whose vertices are delivery coordinates. The matrix entry `adjMatrix[i][j]` is the travel cost from point i to point j , while `coords[i][0]`

and `coords[i][1]` store planar coordinates. The implementation treats non-positive matrix entries as absent edges during relaxation [C4, ll. 156-158]. The path cost is

$$g(n) = \sum_{(u,v) \text{ on the chosen route}} w(u, v).$$

6.2 Three heuristic

The lambda `estimateToTarget` implements three heuristic modes [C4, ll. 98-107]:

$$h_1(n) = |x_n - x_t| + |y_n - y_t| \quad \text{Manhattan,} \quad (4)$$

$$h_2(n) = \sqrt{(x_n - x_t)^2 + (y_n - y_t)^2} \quad \text{Euclidean,} \quad (5)$$

$$h_3(n) = \max(|x_n - x_t|, |y_n - y_t|) \quad \text{Chebyshev.} \quad (6)$$

These metrics are standard geometric lower-bound ideas in path planning; their usefulness depends on whether graph edge weights are compatible with the corresponding geometry [4, 7]. The code correctly implements the requested modes, but it does not verify that the input weights obey geometric lower bounds.

Table 3: Some heuristic modes.

Heuristic	Formula	Best intuition	Risk if weights are arbitrary
Manhattan	$\Delta x + \Delta y$	City-block movement with axis-aligned travel.	May overestimate when diagonal or shortcut edges are cheap.
Euclidean	$\sqrt{\Delta x^2 + \Delta y^2}$	Direct flight or straight-line spatial routing.	May overestimate if edges are discounted below geometric distance.
Chebyshev	$\max(\Delta x, \Delta y)$	Diagonal-dominant movement.	Often conservative, but may be weak on sparse weighted graphs.

6.3 Open-set scan, relaxation, and reconstruction

Task 2 maintains `alreadyUsed`, `waitingRoom`, `routeCost`, and `cameFrom` arrays [C4, ll. 118-130]. It selects a candidate by scanning all 100 vertices and comparing $f = g + h$ under a custom tie-breaker [C4, ll. 109-112, 132-150]. Relaxation then tries every potential neighbor in the matrix row and updates `routeCost` and `cameFrom` if a smaller g -value is found [C4, ll. 156-166]. Finally, the code follows `cameFrom` backward from the goal and builds a linked list with coordinate names [C4, ll. 169-184].

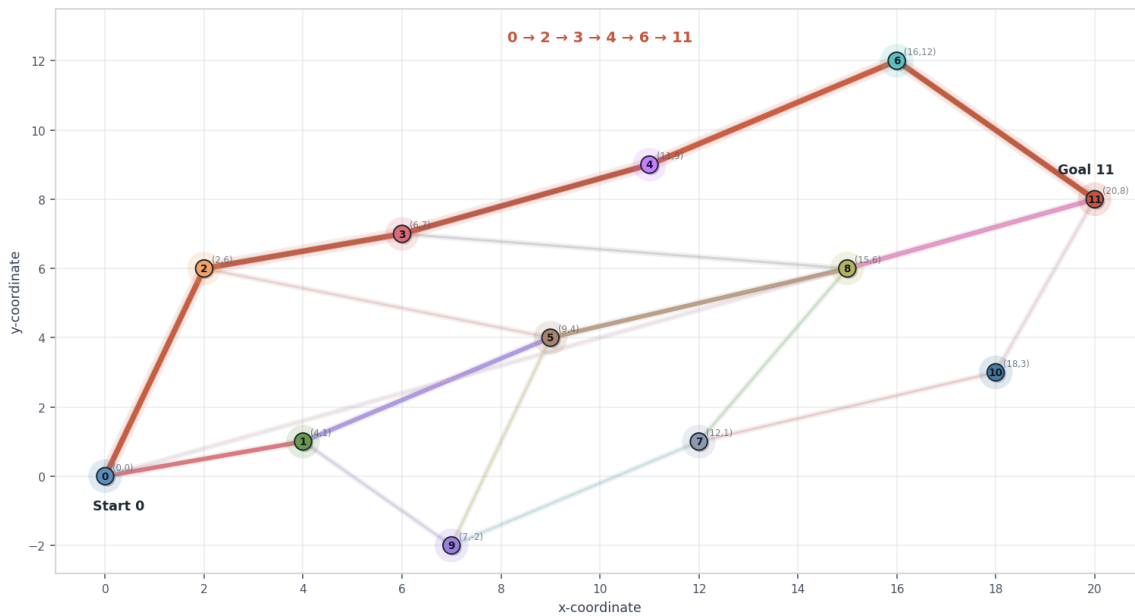


Figure 3: Aurora relay map from main.cpp

6.4 Test case: Aurora relay

The modified driver defines twelve drone waypoints and weighted edges [C5, ll. 137-182]. It then runs all three heuristic modes from waypoint 0 to waypoint 11 [C5, ll. 184-190]. The resulting paths in [T1] are summarized in Table 4.

Table 4: Validation outputs from the Aurora relay scenario

Mode	Returned path	Final g	Interpretation
Manhattan	$(0, 0) \rightarrow (2, 6) \rightarrow (6, 7) \rightarrow (11, 9) \rightarrow (16, 12) \rightarrow (20, 8)$	14.20	Axis-aligned urgency biases the search through upper waypoints.
Euclidean	$(0, 0) \rightarrow (4, 1) \rightarrow (9, 4) \rightarrow (15, 6) \rightarrow (20, 8)$	11.80	Straight-line guidance finds the lower-cost spatial route.
Chebyshev	$(0, 0) \rightarrow (4, 1) \rightarrow (9, 4) \rightarrow (15, 6) \rightarrow (20, 8)$	11.80	A conservative diagonal metric agrees with Euclidean here.

This example makes the report more interesting than a synthetic four-node graph: the same weighted map produces different search personalities. The Manhattan heuristic returns a valid route, but not the cheapest among the displayed outputs. This is consistent with the admissibility warning: when edge weights do not obey Manhattan lower bounds, Manhattan guidance can be misleading.

6.5 Unreachable case

The driver also constructs a target that exists in the coordinate system but is disconnected from the air network [C5, ll. 192-212]. The function returns `nullptr`; the displayed result is “No path found” in [T1]. This validates the failure path at [C4, l. 149].

7 Task 3: Warehouse Robot Navigation with Obstacles

7.1 Model

Task 3 uses a grid where 0 denotes a passable cell and 1 denotes an obstacle. The robot can move in eight directions. Cardinal moves cost 1, and diagonal moves cost 1.5. The movement table is explicitly encoded in the array `walk` [C4, ll. 235-243]. The heuristic is Manhattan when `mode == 1` and Chebyshev otherwise [C4, ll. 213-217].

7.2 Implementation-level analysis

The function validates grid size, coordinate bounds, blocked start/goal, and the start-equals-goal condition [C4, ll. 230-233]. It then initializes `bestG`, `closed`, `parentX`, `parentY`, and `takenMove` [C4, ll. 245-263]. Candidate selection scans a vector frontier and uses `comesFirst` to choose the smaller f , with deterministic tie-breaking by row, column, and cost [C4, ll. 219-228, 265-278]. Neighbor relaxation updates parent coordinates and the textual move label [C4, ll. 280-294]. This explains why Task 3 linked-list node names are directions rather than coordinates: `takenMove[nx][ny]` is exactly the label used when entering a cell [C4, ll. 289-291, 319-325].

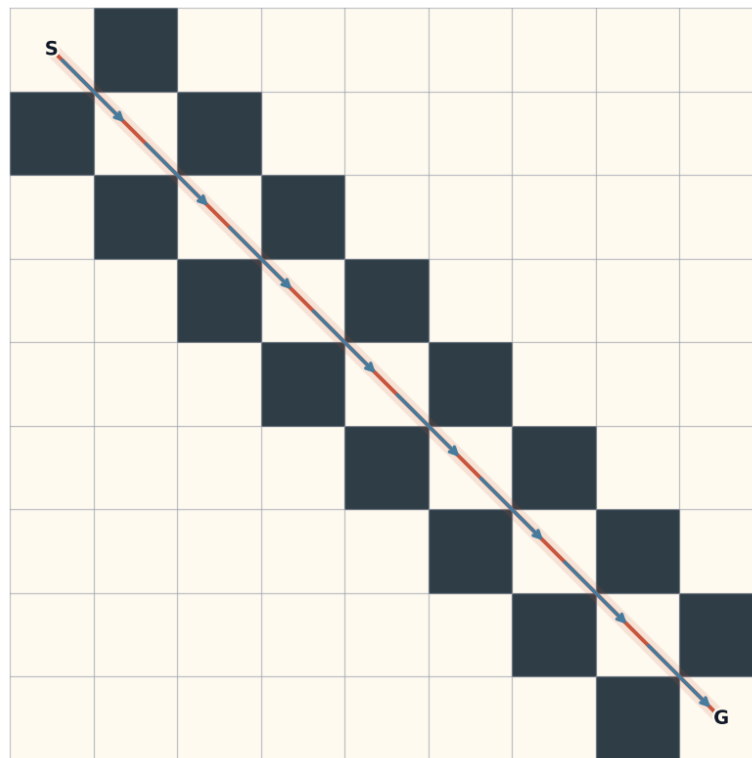


Figure 4: Diagonal keyhole warehouse with obstacles block ordinary horizontal and vertical corridors, but diagonal movement threads a path from S to G

7.3 Test case: Diagonal keyhole

The modified driver defines a nine-by-nine “diagonal keyhole” layout [C5, ll. 219-238]. Both Manhattan and Chebyshev modes return eight consecutive **Down-Right** moves. Table 5 compares the cost values shown in [T1].

Table 5: Both heuristics choose the same physical route, but assign different h and f values.

Mode	Move sequence	Final g	Initial displayed f	Observation
Manhattan	Eight Down-Right moves	12.00	15.50	Overestimates the remaining diagonal route early.
Chebyshev	Eight Down-Right moves	12.00	8.50	More compatible with diagonal progress, but still not exact under diagonal cost 1.5.

The test is pedagogically strong because it dramatizes why diagonal movement changes heuristic suitability. Manhattan thinks a diagonal target is far in two axes; Chebyshev sees the diagonal alignment.

7.4 Test case: Sealed freezer vault

The driver also defines a layout where the start is open but boxed in [C5, ll. 240-252]. The result is correctly reported as “No path found” in [T1]. This validates obstacle checking and the final no-route return at [C4, l. 297].

8 Task 4: Evacuation Route Planning

8.1 Model and grid-to-matrix transformation

Task 4 differs from Task 3 in representation. It first converts a floor plan grid into a weight matrix, then applies A* over flat vertex IDs. The conversion uses

$$\text{vertex_id} = x \cdot n + y. \quad (7)$$

The helper `buildWeightMatrix` clears the full matrix, validates $m \cdot n \leq 100$, skips blocked cells, and creates weighted edges to passable neighbors [C4, ll. 340-372]. A matrix value of zero means no direct edge, a cardinal move has weight 1, and a diagonal move has weight 1.5 [C4, ll. 363-368].

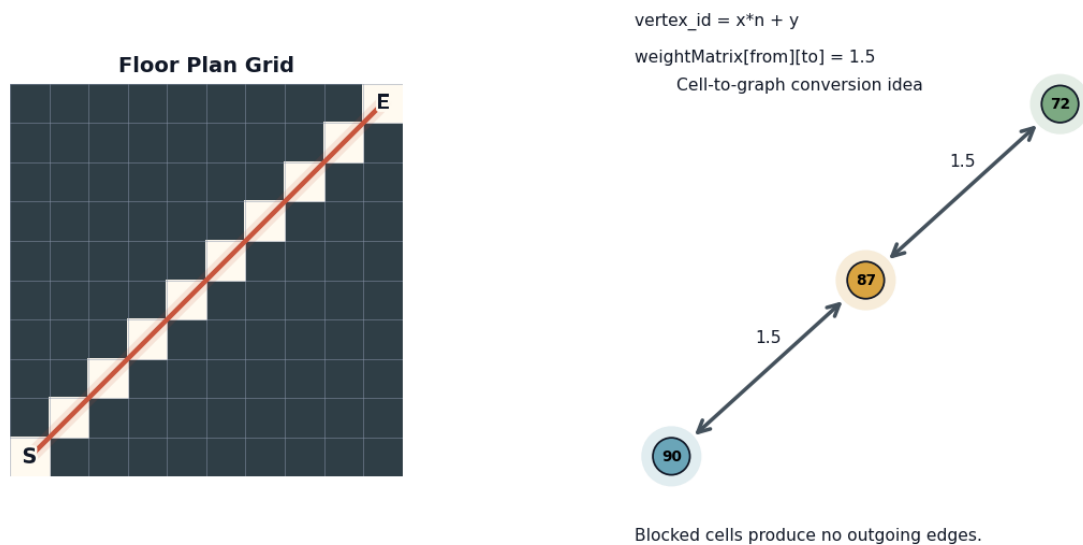


Figure 5: The left side shows the fire-door diagonal corridor; the right side sketches how cells become numbered vertices and diagonal edges obtain weight 1.5.

8.2 A* over vertex indices

The evacuation heuristic function is shared by both modes: Manhattan if `mode == 1`, Chebyshev otherwise [C4, ll. 374-379]. The main function validates dimensions and blocked start/exit cells [C4, ll. 381-392], initializes arrays `gScore`, `fScore`, `parent`, `opened`, and `finished` [C4, ll. 397-413], scans for the open vertex with minimum f [C4, ll. 415-424], and relaxes all nonzero matrix neighbors [C4, ll. 429-443]. It then reconstructs the route and creates coordinate-labeled `PathNode` objects [C4, ll. 446-470].

8.3 Test cases

The modified driver includes three Task 4 cases. First, the fire-door staircase is a narrow diagonal escape shaft across a 10×10 floor [C5, ll. 259-280]. Both Manhattan and Chebyshev modes return the diagonal corridor from $(9, 0)$ to $(0, 9)$, with total cost 13.5, matching nine diagonal steps at cost 1.5 each. Second, the smoke-ring floor places the exit visibly inside a wall ring; the function returns no path [C5, ll. 282-296]. Third, the already-at-exit sanity check returns a single node $(0, 0)$ with $f = g = h = 0$ [C5, ll. 298-308]. This contrasts with Task 3, where start-equals-goal returns `nullptr` by design [C4, l. 233].

Table 6: Task 4 validation

Scenario	Expected behavior	Observed output	Interpretation
Fire-door staircase	Diagonal corridor path	Cost 13.50 in both modes	Conversion preserves diagonal movement with weight 1.5.
Smoke-ring floor	No path	No path found	Blocked-cell exclusion works.
Already-at-exit	Zero movement path	Single node $(0, 0)$	Task 4 handles the trivial target case.

9 Comparative Heuristic Discussion

Table 7: Comparison of all four task models.

Task	Graph model	Edge cost	Heuristic	Output label	Main challenge
1	Social adjacency matrix	Unit social hop	Exact reverse-BFS distance	Person ID	Turning degree-of-separation into a shortest-hop problem.
2	Weighted coordinate graph	Input matrix	Manhattan, Euclidean, Chebyshev	Coordinate string	Choosing a heuristic compatible with arbitrary weights.
3	Warehouse grid	Cardinal 1, diagonal 1.5	Manhattan or Chebyshev	Direction name	Obstacle-aware movement and direction reconstruction.
4	Floor-plan matrix	Generated 1 or 1.5	Manhattan or Chebyshev	Coordinate string	Correct grid-to-graph conversion.

The central lesson is that A* is not merely an algorithmic template; it is a modeling discipline. A heuristic must be interpreted against the edge-cost semantics. An elegant distance formula can be harmful if it overestimates. Conversely, a weaker but safe heuristic can preserve optimality while reducing less of the search space. This is exactly the tradeoff discussed in classical heuristic search theory [1, 6, 4].

10 Correctness Argument

10.1 Validity of returned paths

For Tasks 2-4, every returned path is constructed from parent pointers assigned only during edge relaxation. In Task 2, `cameFrom[next] = chosen` is executed only when a positive matrix edge is relaxed [C4, ll. 156-164]. In Task 3, parent coordinates and move labels are updated only after a legal in-grid, unblocked neighbor is found [C4, ll. 280-292]. In Task 4, `parent[next] = current` is assigned only when `weightMatrix[current][next]` is nonzero [C4, ll. 429-440]. Therefore, every consecutive pair in the reconstructed output corresponds to a legal edge or movement.

10.2 Optimality under standard assumptions

Under non-negative edge costs and admissible, consistent heuristics, A* returns an optimal path [1, 4]. Task 1 satisfies these assumptions because reverse BFS computes the exact remaining hop distance. Task 2 satisfies them only when selected geometric heuristics are lower bounds on actual edge costs. Task 3 and Task 4 satisfy them for Chebyshev-like conservative estimates more plausibly than for Manhattan, but the diagonal cost 1.5 means the ideal grid heuristic would be closer to an octile-style metric:

$$h_{octile}(n) = \max(\Delta x, \Delta y) + 0.5 \min(\Delta x, \Delta y). \quad (8)$$

This formula is not in the submitted assignment signature, so the code correctly implements the requested modes while the report honestly records the theoretical limitation.

10.3 Implementation limitations affecting proofs

The code uses zero to mean no edge, so zero-cost edges are not representable. It also uses fixed arrays of size 100. These choices simplify compatibility with the assignment interface but prevent a general mathematical statement over arbitrary graph sizes. Finally, Task 2's tie-breaking in `shouldTakeVertex` has a special opening condition [C4, ll. 109-112]. This does not break validity, but it can influence which equal-priority route is returned.

11 Complexity Analysis

Let V be the number of vertices and E the number of edges. With a priority queue, a sparse-graph A* implementation can be close to $O((V + E) \log V)$ under standard assumptions [3]. The submitted implementation instead scans arrays and vectors manually, so the worst-case behavior is quadratic for most tasks.

Table 8: Complexity of the implementation

Task	Main operations	Time	Space
1	Reverse BFS-like matrix scan plus greedy descent	$O(V^2)$	$O(V)$ auxiliary plus matrix input.
2	Scan 100 vertices for selection and scan matrix row for relaxation	$O(V^2)$	$O(V)$ auxiliary plus matrix input.
3	Linear vector-frontier selection; at most 8 neighbor checks per cell	$O(V^2)$ worst case	$O(V)$.
4	Matrix construction and matrix-based A* scan	$O(V^2)$	$O(V^2)$ for <code>weightMatrix</code> .

For the assignment's 100-node constraint, this design is sufficient and readable. For real navigation systems, a priority queue, adjacency lists, and dynamic graph sizes would be more appropriate.

12 Testing and Demonstration

The validation program is an important part of the submission because it demonstrates normal routes, heuristic variation, unreachable targets, obstacle traps, and boundary behavior. Table 9 maps test scenarios to code ranges and expected outcomes.

Table 9: Validation scenarios from the `main.cpp`.

Scenario	Code citation	Expected output	Test cases
Friend bridge	[C5, ll. 100-130]	$0 \rightarrow 2 \rightarrow 4$	Exact social heuristic and linked-list output.
Aurora relay	[C5, ll. 137-190]	Three heuristic-dependent routes	Heuristic behavior under same weighted graph.
Blackout island	[C5, ll. 192-212]	No path found	Disconnected target handling.
Diagonal keyhole	[C5, ll. 219-238]	Eight diagonal moves	Diagonal movement with obstacles.
Sealed freezer vault	[C5, ll. 240-252]	No path found	Start boxed in by obstacles.
Fire-door staircase	[C5, ll. 259-280]	Diagonal evacuation route	Grid-to-matrix conversion.
Smoke-ring floor	[C5, ll. 282-296]	No path found	Exit sealed by walls.
Already-at-exit	[C5, ll. 298-308]	Single zero-cost node	Trivial evacuation case.

The exact console output is stored in `test_cases/expected_output_from_main.txt`. An excerpt is included in Appendix ??.

13 Limitations and Future Improvements

13.1 Limitations

- The code uses fixed-size arrays and therefore cannot handle graphs beyond the assignment's 100-node interface.
- The implementation uses linear scans rather than `std::priority_queue`; this is simple but not asymptotically optimal.
- Some heuristics are not guaranteed admissible under arbitrary edge weights or diagonal movement costs.
- Manual memory management is used for `PathNode`; callers must free the linked list.
- Task 3 treats start-equals-goal as no path, while Task 4 returns a zero-cost path. This is a documented behavioral difference.

13.2 Future work

A stronger version would use adjacency lists and priority queues, support dynamic graph sizes, add a mathematically tighter octile heuristic for eight-direction grids, generate figures automatically from the input graphs, and include a unit-test harness comparing returned path costs against Dijkstra baselines.

14 Conclusion

This report transformed the A* assignment into a case study of heuristic search. The four tasks show that the same equation, $f(n) = g(n) + h(n)$, becomes meaningful only after careful modeling. In the social task, the heuristic is exact. In the drone task, the heuristic behaves like a navigation personality. In the warehouse task, diagonal motion reveals the danger of mismatched heuristics. In evacuation planning, grid-to-graph conversion makes indoor routing explicit.

References

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968, doi: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136).
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, Dec. 1959, doi: [10.1007/BF01386390](https://doi.org/10.1007/BF01386390).
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022. [Online]. Available: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- [4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021. [Online]. Available: <https://aima.cs.berkeley.edu/>
- [5] R. Diestel, *Graph Theory*, 5th ed. Berlin, Germany: Springer, 2017. [Online]. Available: <https://diestel-graph-theory.com/>
- [6] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA, USA: Addison-Wesley, 1984. [Online]. Available: https://mat.uab.cat/~alseda/MasterOpt/Judea_Pearl-Heuristics_Intelligent_Search_Strategies_for_Computer_Problem_Solving.pdf
- [7] A. Patel, “Introduction to the A* Algorithm,” Red Blob Games, 2025. [Online]. Available: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>. [Accessed: May 27, 2026].